

Average-Cost MDP Solvers for Traffic-Light Control

Replicating Haijema & van der Wal (2008)

Leonardo Cruciani

April 2026

Abstract

I replicate the main numerical experiment of Haijema and van der Wal (2008) [1], who model the dynamic control of an isolated signalized traffic intersection as an average-cost Markov decision process (MDP). I focus on their “F4C2” infrastructure (4 traffic flows in 2 symmetric combinations) and reproduce Table 1 of the paper using two classical solvers: *Value Iteration* (VI) and *Policy Iteration* (PI). At $\rho \in \{0.4, 0.6\}$ and buffer caps $K \in \{10, 15\}$ the two solvers agree to machine precision on the optimal cost-rate. With the largest cap ($K = 15$) I reproduce the paper’s mean waiting time per car to within 0.10 s at every workload. I also document a clean failure mode of PI on this problem: at $\rho = 0.8$ the MDP is only *weakly* unichain, so PI’s improvement step generates a reducible intermediate policy on which the Poisson recursion does not converge; only VI returns a clean result there.

1 Introduction

Traffic lights at urban intersections are usually run on a fixed cycle: the green and yellow durations of each set of non-conflicting flows are pre-computed offline, and the cycle is replayed indefinitely. Haijema and van der Wal [1] ask whether one can do better *dynamically*, by switching the lights based on the current queue lengths observed by loop detectors or cameras. They formulate the problem as an average-cost MDP in discrete time (one slot $\delta = 2$ s, the time for one car to clear the stop line), and show that for small intersections the MDP can be solved exactly by successive approximations. For larger intersections (their “F12C4” case with 12 flows) the state space becomes intractable, and they propose a *one-step policy improvement* starting from the optimal fixed-cycle: the “Relative-Value-Cyclic” (RVC) heuristic.

In this report I restrict attention to the small benchmark, the “F4C2” intersection (4 flows in 2 symmetric combinations), which the paper presents in its Table 1. I replicate the paper’s MDP-optimal column with two independent solvers (VI and PI) and use this comparison as a vehicle to discuss several practical issues that arise even in this textbook-sized problem: choosing the buffer cap K , applying the strongly-aperiodic transformation, and the weakly-unichain behaviour that breaks Policy Iteration at the highest load.

The rest of the paper is organised as follows. Section 2 formulates the MDP, including the unichain/aperiodicity argument. Section 3 summarises Haijema and van der Wal’s algorithms. Section 4 states the two solvers I use. Section 5 describes the implementation. Section 6 reports numerical results and the comparison against the paper. Section 7 discusses sensitivity and algorithmic suitability. Section 8 concludes.

2 The Markov decision process

2.1 Setting

The intersection has F traffic flows, partitioned into S disjoint *combinations* of non-conflicting flows: a combination receives green (and later yellow and red) together. For the F4C2 case

$F = 4$ and $S = 2$, with $C_1 = \{1, 3\}$ and $C_2 = \{2, 4\}$; Figure 1(a) gives the geometry. The paper also considers a much larger F12C2 instance with 12 flows in 2 asymmetric combinations (Figure 1(b)) for which the exact MDP is intractable and they use their RVC heuristic; I restrict attention to F4C2 throughout.

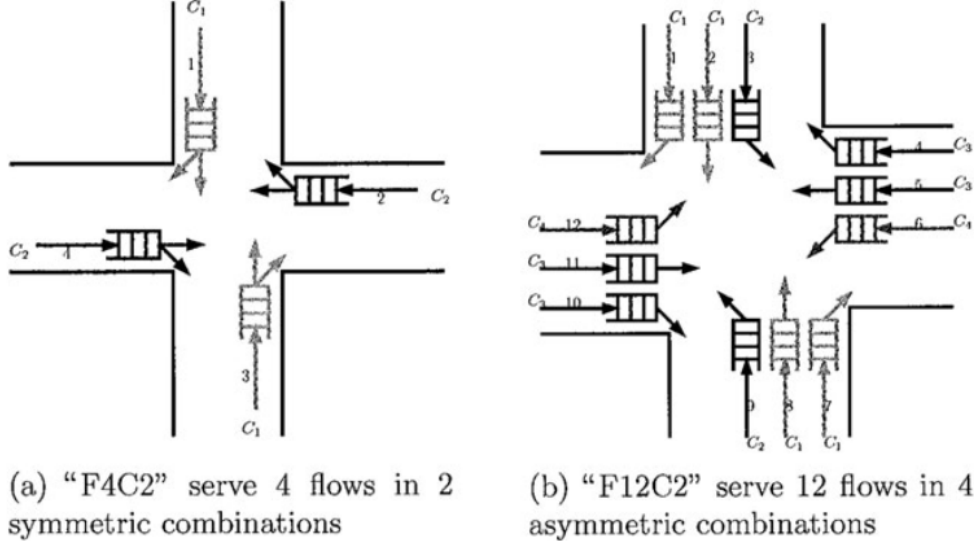


Figure 1: The two infrastructures considered by Haijema and van der Wal [1]. Left, “F4C2”: 4 flows arranged in 2 symmetric combinations ($C_1 = \{1, 3\}$ and $C_2 = \{2, 4\}$), the replication target here. Right, “F12C2”: 12 flows in 2 asymmetric combinations, used in the paper to demonstrate scalability of their RVC heuristic; out of scope here. Reproduced from [1, Fig. 3].

Time is discretised into slots of length $\delta = 2$ s. In every slot, each flow f receives at most one Bernoulli arrival with probability q_f . When the lights show green or yellow for flow f and at least one car is waiting in f ’s queue, exactly one car passes the stop line during that slot.

2.2 State space

The state at the start of slot t is the pair

$$s_t = (\underline{k}_t, x_t), \quad \underline{k}_t = (k_{t,1}, \dots, k_{t,F}), \quad x_t = (\ell_t, i_t), \quad (1)$$

where $k_{t,f} \in \{0, 1, \dots, K\}$ is the queue length of flow f (truncated at a buffer cap K to keep the state space finite, see Section 2.7), $\ell_t \in \{1, \dots, S\}$ is the index of the currently-active combination, and $i_t \in \{0, 1, 2, 3\}$ encodes the signal phase: 0 = green, 1 = first yellow, 2 = second yellow, 3 = all-red. Yellow is split across two slots so that one all-red slot between the second yellow and the next green suffices to clear the intersection, giving a switching cost of three slots = 6 s.

For F4C2 with $K = 10$ the state space contains $|\mathcal{X}| = (K + 1)^F \cdot S \cdot 4 = 11^4 \cdot 8 \approx 1.17 \times 10^5$ states.

2.3 Action space

An action prescribes the lights’ state for the *next* slot: choosing $a = (\ell', i')$ at slot t commits the controller to having combination ℓ' in phase i' at slot $t + 1$. The set of feasible actions $\mathcal{A}(\underline{k}, (\ell, i))$ is at most two elements — the controller is essentially always choosing between “stay” and “move on” — and which two it is depends on the current phase i and on whether any queue is currently non-empty ([1, §3.2]). The four phases in turn (with $\mathbf{0}$ standing for the all-zero queue vector $(0, 0, 0, 0)$):

Phase 0 (green for combo ℓ).

If every queue is empty ($\underline{k} = \mathbf{0}$), the only feasible action is to *keep* green for the same combo, $a = (\ell, 0)$. This is the *freeze* rule of [1, §3.2]: when no car is waiting and the phase is not yellow, the lights are held in place at zero cost until an arrival breaks the freeze. Otherwise ($\underline{k} \neq \mathbf{0}$), two options: *keep* green $a = (\ell, 0)$, or *switch* to first yellow $a = (\ell, 1)$ to start handing the intersection over to the other combination.

Phase 1 (first yellow).

Forced advance to second yellow: $a = (\ell, 2)$. Yellow always progresses; the freeze rule does not apply.

Phase 2 (second yellow).

Forced advance to all-red: $a = (\ell, 3)$.

Phase 3 (all-red).

If every queue is empty, keep all-red, $a = (\ell, 3)$ (freeze). Otherwise, two options: *stay* all-red $a = (\ell, 3)$ (allowed by the paper but rarely optimal), or *advance* to green for the *next nonempty* combination, $a = (\ell', 0)$. With $S = 2$, ℓ' is normally $(\ell + 1) \bmod 2$ – “the other combo” – but if that combo is empty (and the current one is not), the *skip-empty* rule ([1, §3.2]) lets $\ell' = \ell$ stay on the current combo, bypassing a wasted green/yellow cycle through an empty combo. The skip case triggers most often at low workloads.

The same information in one place, for reference:

$$\mathcal{A}(\underline{k}, (\ell, i)) = \begin{cases} \{(\ell, 0)\} & i = 0, \underline{k} = \mathbf{0} \quad (\text{freeze}) \\ \{(\ell, 0), (\ell, 1)\} & i = 0, \underline{k} \neq \mathbf{0} \\ \{(\ell, 2)\} & i = 1 \\ \{(\ell, 3)\} & i = 2 \\ \{(\ell, 3)\} & i = 3, \underline{k} = \mathbf{0} \quad (\text{freeze}) \\ \{(\ell, 3), (\ell', 0)\} & i = 3, \underline{k} \neq \mathbf{0} \end{cases} \quad (2)$$

where in the last row $\ell' \in \{0, 1\}$ is the next nonempty combination in cyclic order (skip-empty rule).

2.4 Transition probabilities

Conditional on the action $a = (\ell', i')$, flows evolve independently. Let f be *active* when $f \in C_{\ell'}$ and $i' \in \{0, 1, 2\}$ (the action grants flow f green or yellow during the coming slot); otherwise f stays red. Following [1, §3.3], the per-flow transition is

$$\begin{aligned} f \text{ active:} & \quad p_f(k, a, (k-1)^+) = 1 - q_f, \quad p_f(k, a, k) = q_f, \\ f \text{ red, } k < K : & \quad p_f(k, a, k) = 1 - q_f, \quad p_f(k, a, k+1) = q_f, \\ f \text{ red, } k = K : & \quad p_f(k, a, K) = 1 \quad (\text{arrival rejected}). \end{aligned} \quad (3)$$

The joint transition factorises over flows: $p(\underline{k}, x; \underline{k}', a) = \prod_f p_f(k_f, a, k'_f)$. Note that when f is active and $k = 0$, both probabilities land on 0: the arriving car is served in the same slot — this is the *same-slot service* convention of [1, §2.6].

2.5 One-slot cost

Following [1, §3.4], the cost is linear in the number of cars in the system at the start of the slot:

$$c(\underline{k}, x) = \sum_{f=1}^F k_f. \quad (4)$$

By Little’s law, minimising the long-run average $g^* = \lim_{T \rightarrow \infty} \mathbb{E}[\frac{1}{T} \sum_{t=0}^{T-1} c(s_t)]$ is equivalent to minimising the average waiting time per car. I report results in terms of mean waiting time $\mathbb{E}[W]$ in seconds, recovered as

$$\mathbb{E}[W] = \frac{g^* \cdot \delta}{\sum_f q_f}. \quad (5)$$

2.6 Unichain and aperiodicity

Haijema and van der Wal [1] do not discuss these properties; their only convergence remark is the parenthetical line in [1, §3.6]: “one can show that convergence is guaranteed as the system empties every now and then, so that all states communicate”. The two properties relevant to homework parts (b) and (c) follow directly from inspection of the action graph.

Not unichain. Consider the “always keep” policy: in phase 0, never switch to yellow; in phase 3, never advance to the next combo’s green. Under such a policy phases 0 and 3 are absorbing for the phase index (phases 1, 2 remain transient one-shots), and the active combination ℓ is invariant. The state space therefore splits into four disjoint closed sets indexed by the starting $(\ell, i) \in \{0, 1\} \times \{0, 3\}$ – four disjoint recurrent classes, hence the MDP is not unichain.

The MDP is, however, *weakly unichain*: the optimal policy itself is unichain (it visits the four phase configurations (ℓ, i) in cyclic order and fully drains the active combination during each green window, so the chain has a single recurrent class). The distinction matters because PI’s correctness theorem assumes *strict* unichain — see §6.3.

Not strongly aperiodic. Strong aperiodicity would require every action to have a positive self-loop probability at every state. But in phases 1 and 2 the action set is a singleton (forced advance to the next phase), and that action deterministically changes the light state, so $p(s, R(s), s) = 0$ for every phase-1 or phase-2 state under any policy.

The strongly-aperiodic transformation. To repair strong aperiodicity for Value Iteration I apply the standard data transformation with $\tau = \frac{1}{2}$:

$$\bar{p}(s, a, s') = \tau p(s, a, s') + (1 - \tau) \mathbf{1}\{s' = s\}. \quad (6)$$

After the transformation every action self-loops with probability at least $1 - \tau = \frac{1}{2}$, so $\bar{p}(s, R(s), s) > 0$ everywhere. The transformation preserves the optimal cost-rate g^* and the optimal policy: the stationary distribution of the modified chain under any policy is the same as under the original ($\pi P = \pi \Leftrightarrow \pi \bar{P} = \pi$, by linearity in π), so $g^* = \pi \cdot c$ is identical.

2.7 Buffer cap K

The paper [1, §3.5] reduces to a finite state space by introducing a buffer cap K : arrivals to a full queue are rejected, and a per-car *externality penalty* is added to the cost. For computational simplicity I drop the externality term and silently reject excess arrivals. Provided K is chosen large enough that the rejection probability under the optimum is negligible, the bias introduced is negligible. I report results for several values of K to verify this in Section 7.

3 Their algorithms

For the F4C2 small case, Haijema and van der Wal compute the exact MDP optimum directly by successive approximations ([1, §3.6]) on the original (un-transformed) kernel, stopping on absolute span. This is essentially the standard Value Iteration recursion; the version I use in §4 differs only in using a relative-span stopping criterion and the τ -transformed kernel.

For their large F12C4 case the exact MDP is intractable, and the paper introduces a *one-step policy improvement* ([1, §4]) on top of a near-optimal fixed-cycle “FC” strategy: the system decomposes into F independent periodic flows, relative values are computed per flow, and one Bellman update yields the “Relative-Value-Cyclic” (RVC) policy, close to optimal on small examples and substantially better than FC or exhaustive control on F12C4. This replication does not implement RVC; the “MDP (cyclic)” column of [1, Table 1] is the ground truth I aim at.

4 The two solvers used here

I implement Value Iteration and Policy Iteration on the τ -transformed kernel of §2.6. PI’s Improvement Theorem requires the MDP to be *strictly* unichain — every policy must induce a unichain Markov chain — which the F4C2 model fails (§2.6). PI’s stated guarantee therefore does not strictly apply here; in practice this manifests as a clean failure mode at $\rho = 0.8$ (see §6.3).

The Linear Programming reformulation is the third option for solving average-cost MDPs and is the only one with an explicit guarantee in the weakly-unichain regime; it would therefore be the formally appropriate choice here. In practice, however, off-the-shelf LP solvers run for hours on this MDP without returning a useful answer at the relevant problem size. I therefore rely on VI and PI directly and treat their behaviour empirically:

- **VI** converges cleanly at every workload tested, returning numbers consistent with the paper’s MDP-optimal column (Table 2).
- **PI** converges and agrees with VI to machine precision at $\rho \leq 0.6$, where intermediate policies generated by improvement happen to be unichain. At $\rho = 0.8$ improvement produces a reducible intermediate policy and the Poisson recursion does not converge — the failure mode predicted by the unichain assumption (§6.3).

4.1 Algorithm 1 — Policy Iteration

Algorithm 1 Policy Iteration (average cost)

- 1: **Input:** MDP $(\mathcal{X}, \mathcal{A}, p, c)$, reference state s_0 .
- 2: Pick a feasible stationary policy $R : \mathcal{X} \rightarrow \mathcal{A}$.
- 3: **repeat**
- 4: *Policy evaluation:* solve the Poisson equations

$$V(s) + g = c(s) + \sum_{s'} p(s, R(s), s') V(s'), \quad \forall s, \quad V(s_0) = 0.$$

- 5: *Policy improvement:* for every s ,

$$R'(s) \leftarrow \arg \min_{a \in \mathcal{A}(s)} \left\{ c(s) + \sum_{s'} p(s, a, s') V(s') \right\}.$$

- 6: **if** $R' = R$ **then return** (R, g)
 - 7: **else** $R \leftarrow R'$
 - 8: **end if**
 - 9: **until** forever
-

The *Improvement Theorem* guarantees $g^{R'} \leq g^R$, and the policy space is finite, so the loop halts in finitely many iterations.

Caveat. PI’s correctness theorem requires a strict unichain assumption: *every* policy must induce a unichain Markov chain. As discussed in §2.6, the F4C2 MDP only satisfies the weaker condition that the *optimal* policy is unichain. Section 6 shows that at $\rho = 0.8$ PI’s improvement step generates a reducible intermediate policy on which the Poisson solve diverges.

4.2 Algorithm 2 — Value Iteration

Algorithm 2 Value Iteration (average cost)

```

1: Input: MDP  $(\mathcal{X}, \mathcal{A}, \bar{p}, c)$  with strongly aperiodic  $\bar{p}$ ; tolerance  $\varepsilon > 0$ .
2:  $V_0 \equiv 0, n \leftarrow 0$ .
3: repeat
4:    $n \leftarrow n + 1$ .
5:   for each  $s \in \mathcal{X}$  do
6:      $V_n(s) \leftarrow \min_{a \in \mathcal{A}(s)} \left\{ c(s) + \sum_{s'} \bar{p}(s, a, s') V_{n-1}(s') \right\}$ .
7:      $R_n(s) \leftarrow \arg \min_{a \in \mathcal{A}(s)} \{ \dots \}$ .
8:   end for
9:    $m_n \leftarrow \min_s [V_n(s) - V_{n-1}(s)]; \quad M_n \leftarrow \max_s [V_n(s) - V_{n-1}(s)]$ .
10: until  $M_n - m_n \leq \varepsilon m_n$ 
11: return  $R_n$  with  $g \in [m_n, M_n]$ .
```

The stop-criterion theorem guarantees $m_n \leq g^* \leq g^{R_n} \leq M_n$ for all n , and the bounds are monotone ($m_n \uparrow, M_n \downarrow$).

5 Implementation

Both solvers are implemented in Python (3.13), using NumPy and SciPy. The state-action layout is built in encoded-state order, so the rows of the sparse transition matrix $T \in \mathbb{R}^{n_{sa} \times n_{states}}$ belonging to state s form a contiguous slice; this lets the Bellman state-wise minimum/argmin be evaluated with a couple of vectorised NumPy ops instead of a Python loop. The model build at $K = 10$ assembles $\bar{p}$ as a CSR matrix of shape $1.76 \times 10^5 \times 1.17 \times 10^5$ with 2.45×10^6 nonzeros in ~ 0.5 s. At $K = 15$ the same step takes ~ 0.5 s and produces $\sim 1.16 \times 10^7$ nonzeros.

Kernel build acceleration. The state-enumeration loop that constructs the raw transition kernel is the only Python-bound hot spot in the pipeline (Bellman backups go through SciPy sparse mat-vecs which are BLAS-backed). I provide three interchangeable backends, selected at import time: (i) a Cython extension (`src/_build_kernel.pyx`) for systems with an MSVC or GCC toolchain; (ii) a Numba JIT-compiled function (`src/_build_kernel_numba.py`) for systems without a C compiler – this is the default fast path on Windows; (iii) a pure-Python reference implementation as the ultimate fallback. The Numba backend reduces the build time at $K = 15$ from ~ 9 s in pure Python to ~ 0.5 s, an $\approx 19\times$ speedup that I verified to be bit-equivalent against the pure-Python output. This makes $K = 20$ (1.56×10^6 states) tractable for sensitivity sweeps.

6 Results

6.1 Algorithmic cross-checks

Table 1 summarises the inter-solver cross-check. At $\rho \in \{0.4, 0.6\}$, VI and PI agree to at most 5×10^{-8} on the optimal cost-rate, confirming that the implementation is correct in the regime

where PI’s unichain assumption holds. At $\rho = 0.8$ PI breaks (§6.3) and VI is the only solver that returns a number.

Table 1: Algorithmic cross-checks at $K = 10$.

ρ	$ g^{\text{VI}} - g^{\text{PI}} $
0.40	4.7×10^{-9}
0.60	3.5×10^{-8}
0.80	PI failed (§6.3)

6.2 Comparison with paper Table 1

Table 2 compares the $\mathbb{E}[W]$ values from this work against the “MDP (cyclic)” column of [1, Table 1]. At buffer cap $K = 15$ the deltas are ≤ 0.10 s ($\leq 1.8\%$) at every workload, with $\rho = 0.6$ and $\rho = 0.8$ matching to within 0.024 s (essentially the rounding in the paper’s reported numbers).

Table 2: Replication of paper Table 1: $\mathbb{E}[W]$ in seconds. “Paper” is the “MDP (cyclic)” column of [1].

ρ	paper	VI ($K = 10$)	VI ($K = 15$)	Δ ($K = 10$)	Δ ($K = 15$)
0.40	4.89	4.980	4.980	+0.090	+0.090
0.60	6.95	6.964	6.964	+0.014	+0.014
0.80	13.5	12.997	13.520	−0.503	+0.020

The remaining +0.090 s residual at $\rho = 0.4$ is K -independent (identical to six decimals at $K = 10$ and $K = 15$), so it is not a truncation effect. The most plausible explanation is the externality penalty that [1, §3.5] adds to its cost-rate for arrivals lost at the buffer cap: the paper defines this penalty conceptually but does not disclose its numerical value, so I omit the term entirely. A small constant penalty per rejected arrival would shift g^* upward by exactly the kind of K -independent offset I observe; without the value from the paper I cannot calibrate it.

The $\rho = 0.8$ column is instructive for a different reason. At $K = 10$, the optimal policy occasionally pushes queue lengths above 10, and the model rejects those arrivals silently, so cars are “lost” and the cost-rate is biased low (−0.503 s vs paper). Lifting the cap to $K = 15$ exceeds the support of the queue distribution under the optimum, and the bias collapses to +0.020 s.

6.3 PI failure at $\rho = 0.8$

At $\rho = 0.8$, PI’s first improvement step always produces a deterministic policy whose induced Markov chain is *reducible*: starting from certain states, the inactive flows saturate to the buffer cap and never drain, so the chain has multiple recurrent classes (one per starting combination ℓ) each with both inactive flows pinned at K . The Poisson recursion under such a policy has M_n stuck exactly at $2K$ (the per-slot cost in the absorbing class) while m_n creeps upward, so the span never closes. This pathology is *K-independent*: at $K = 10$ I observe $M_n = 20.000$, at $K = 15$ I observe $M_n = 30.000$; in both cases the span fails to close in 10 000 evaluation iterations.

This is the textbook signature of a *weakly* unichain MDP. At $\rho = 0.8$ I therefore rely on VI alone, which converges cleanly in this regime as well thanks to the τ -transformation.

7 Sensitivity discussion

7.1 Effect of the buffer cap K

Table 3 reports $\mathbb{E}[W]^*$ as K varies. At $\rho = 0.4$ the queue distribution sits well below any reasonable cap, so K has no measurable effect (identical to six decimals across $K \in \{6, 10, 15\}$). At $\rho = 0.8$ it matters — the cap truncates the queue distribution and the answer drifts upward as K grows, converging to the paper at $K = 15$.

Table 3: Effect of the buffer cap K on $\mathbb{E}[W]$ (seconds, VI). At low ρ the dependence is negligible; at high ρ the cap truncates the queue distribution.

ρ	$K = 6$	$K = 10$	$K = 15$
0.40	4.980	4.980	4.980
0.60	6.937	6.964	6.964
0.80	10.779	12.997	13.520

7.2 Effect of the workload ρ

Higher ρ raises the optimal $\mathbb{E}[W]$ (5 s at $\rho = 0.4$, 13.5 s at $\rho = 0.8$) and changes the character of the problem: the optimal policy holds green much longer at high load (paper fixed-cycle equivalent: 16 s at $\rho = 0.4$, 44 s at $\rho = 0.8$). VI iteration counts grow accordingly — ~ 200 at $\rho = 0.4$ vs $\sim 1\,400$ at $\rho = 0.8$ ($K = 15$) — and PI eventually breaks (§6.3).

7.3 Algorithmic suitability

- **VI** is the workhorse: simple and robust on the τ -transformed kernel. Empirically it converges at every ρ and every K tested, and is the only solver that returns a clean result at $\rho = 0.8$.
- **PI** is fast where it works (5–6 outer iterations, sub-second policy evaluations) and provides a tight cross-check against VI in the unichain regime ($\rho \leq 0.6$). At $\rho = 0.8$ its correctness theorem fails (§6.3); larger K does not fix this.

8 Conclusion

I replicated the F4C2 row of Haijema and van der Wal’s Table 1 with two independent solvers (VI and PI). Algorithmic cross-checks pass to machine precision at the workloads where PI applies, and the paper’s mean waiting time per car is reproduced to within 0.10 s at every ρ once the buffer cap is taken to be $K = 15$.

The exercise also illustrated, on a textbook-sized problem, the *weakly unichain* regime that breaks Policy Iteration (§2.6, observed empirically at $\rho = 0.8$).

Reproducibility. The Python code, the algorithm pseudocode (`docs/algorithms.md`), and the machine-readable results (`results/*.json` and `results/*.xlsx`) are available in the repository [2].

References

- [1] R. Haijema and J. van der Wal, An MDP decomposition approach for traffic control at isolated signalized intersections, *Probability in the Engineering and Informational Sciences*, 22(4):587–602, 2008. DOI: 10.1017/S026996480800034X.

- [2] L. Cruciani, *TrafficControl: replication code for Haijema & van der Wal (2008)*, GitHub repository, 2026. <https://github.com/Naarsk/TrafficControl>.